

Cryptography and Network Security

Integrated Situation Report

Masudi Saleh Bertil

September 28, 2026

1 Introduction

A polytechnic institute stores student records on a central server and transfers files between two campuses. A security review found weak staff passwords, outdated software, unencrypted file transfers and guest network access to the records server, and repeated attempts to reach the server from an unfamiliar external address. This report assesses the risks, presents a Python toolkit for encryption and integrity checking, and documents the firewall rules applied in the authorised laboratory.

2 Risk Assessment

2.1 Assets, vulnerabilities and consequences

#	Asset	Vulnerability	Possible consequence
R1	Staff accounts	Weak passwords	Password guessed; unauthorised reading or changing of records
R2	Student records server	Guest network can reach it	Data theft or attack from a guest device
R3	Files sent between campuses	Unencrypted transfers	Interception, reading or tampering

2.2 Ranking

Rank	Risk	Likelihood	Impact	Reason
1	R2 Guest access	High	High	Guests are already inside the network and external probing was observed; the server holds all records.
2	R1 Weak passwords	High	High	Easy to guess; a stolen account looks like a normal user.
3	R3 Unencrypted transfers	Medium	High	Attacker must be on the path, but all data is then readable.

2.3 Recommended controls

R2: firewall rules that block the guest network and allow only authorised staff. R1: strong password policy with multi-factor authentication and account lockout. R3: encrypt files before transfer and verify integrity with SHA-256.

3 Encryption and Integrity Toolkit

The program `src/secure_records.py` uses the Python `cryptography` library. Encryption uses Fernet, which combines AES-128 in CBC mode with an HMAC-SHA256 authentication tag, so a wrong key or a modified file is detected during decryption. The secret key is created by the `keygen` command and stored outside the repository (`~/.exam_keys/records.key`) with permissions 600.

- **encrypt**: reads a file, stores its SHA-256 hash as a baseline, and writes the encrypted output.
- **decrypt**: decrypts a file and, with `--original`, compares its hash to the original to confirm the contents match.
- **hash / check**: `hash` stores a SHA-256 baseline; `check` recalculates it and reports whether the file changed.
- **Error handling**: missing files, empty files, missing keys and wrong input produce a clear ERROR message instead of a crash.

3.1 Test evidence

The output below was produced by `tests/run_tests.sh`.

```
=== 1. Encrypt the sample file ===
Encrypted sample_data/students_sample.csv -> sample_data/students_sample.enc
SHA-256 (original): 8bdf7b0dd93397d07317f740762e9b39eabc1554554fccb1f95dc297a00565fd

=== 2. Decrypt and verify it matches the original ===
VERIFY: MATCH - decrypted file equals original

=== 3. Integrity check on the unchanged file (expect OK) ===
INTEGRITY OK: file unchanged

=== 4. Change a copy of the file, then check (expect FAIL) ===
Baseline stored. SHA-256: 8bdf7b0dd93397d07317f740762e9b39eabc1554554fccb1f95dc297a00565fd
INTEGRITY FAIL: file was modified
    expected 8bdf7b0dd93397d07317f740762e9b39eabc1554554fccb1f95dc297a00565fd
    actual   d97ef5735c0a10c0198ed84c52cf67949e75f8594c08537029c72341de25ce2a

=== 5. Missing file (expect clean error) ===
ERROR: File not found: missing.csv

=== 6. Decrypt a file that is not encrypted (expect clean error) ===
ERROR: Decryption failed: wrong key, or not an encrypted file, or file was changed.

=== 7. Empty file (expect clean error) ===
ERROR: File is empty: tests/empty.txt
```

4 Network Traffic Filtering

4.1 Firewall rules

The rules use `iptables`. Order matters: the guest network is dropped first, then staff are allowed to the service, and finally all other inbound traffic to the service is dropped.

```
#!/bin/bash
# Traffic filtering for the student records server.
# Run as root, ONLY in the authorised laboratory.
```

```

# ---- EDIT THESE VALUES WITH THE ONES GIVEN IN THE LAB ----
SERVER_IP="10.0.10.1" # student records server
GUEST_NET="10.0.20.0/24" # guest network, e.g. 192.168.20.0/24
STAFF_NET="10.0.10.0/24" # authorised staff network, e.g. 192.168.10.0/24
PORT="8080" # service port given by the assessor
PROTO="tcp"
ADMIN_IP="" # optional: your own host, keeps SSH open so you are not locked out
MODE="server" # server = run on the records server; gateway = run on the router
# -----

if [ "$EUID" -ne 0 ]; then echo "Run as root (sudo)."; exit 1; fi
for v in "$GUEST_NET" "$STAFF_NET" "$PORT"; do
    if [ "$v" = "CHANGE_ME" ]; then echo "Edit the values at the top of this script first."; exit
    1; fi
done

iptables-save > /tmp/iptables.backup # backup so you can undo

if [ "$MODE" = "gateway" ]; then
    CHAIN="FORWARD"; DEST="-d_$SERVER_IP"
else
    CHAIN="INPUT"; DEST=""
fi

iptables -F $CHAIN
[ "$CHAIN" = "INPUT" ] && iptables -A INPUT -i lo -j ACCEPT
iptables -A $CHAIN -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
[ -n "$ADMIN_IP" ] && iptables -A $CHAIN -p tcp -s "$ADMIN_IP" $DEST --dport 22 -j ACCEPT

# (a) block guest network to the records server
iptables -A $CHAIN -s "$GUEST_NET" $DEST -j DROP
# (b) allow authorised staff network to the service
iptables -A $CHAIN -p $PROTO -s "$STAFF_NET" $DEST --dport "$PORT" -j ACCEPT
# (c) block all other inbound access to that service
iptables -A $CHAIN -p $PROTO $DEST --dport "$PORT" -j DROP

iptables -L $CHAIN -n -v --line-numbers

```

4.2 Test procedure and results

The tests ran in a virtual laboratory on the laptop (Linux network namespaces with a server and three clients: staff, guest and other), approved by the lecturer. One permitted connection and two blocked connections were tested with `nc -zv -w 3 SERVER PORT` from a staff host, a guest host and a third host. The commands, expected outcomes and actual results are below.

```

# Firewall Filter Tests

## Lab details
- Service and port: TCP port 8080 (fake student records web service)
- Server IP: 10.0.10.1 (staff side), 10.0.20.1 (guest side), 10.0.30.1 (other side)
- Guest network: 10.0.20.0/24
- Staff network: 10.0.10.0/24
- Machine where rules were applied: virtual lab on the laptop (Linux network namespaces), rules
  applied inside the server namespace

## Rules applied
See 'firewall/firewall_rules.sh' and the saved rules in 'firewall/rules.v4'.
Rule order: (a) DROP guest network, (b) ACCEPT staff network to the port, (c) DROP everything
  else to the port.

## Test results (recorded automatically by firewall/test_connection.sh)

```

```

### Test 1: staff host (permitted)
- Run from: bertil-HP-EliteBook-840-G8-Notebook-PC, Mon Sep 28 01:56:55 PM CAT 2026
- Command: nc -zv -w 3 10.0.10.1 8080
- Expected: Connection succeeds
- Actual output: Connection to 10.0.10.1 8080 port [tcp/http-alt] succeeded!
- Actual result: PERMITTED (connection succeeded)

### Test 2: guest host (blocked)
- Run from: bertil-HP-EliteBook-840-G8-Notebook-PC, Mon Sep 28 01:56:58 PM CAT 2026
- Command: nc -zv -w 3 10.0.20.1 8080
- Expected: Connection times out
- Actual output: nc: connect to 10.0.20.1 port 8080 (tcp) timed out: Operation now in progress
- Actual result: BLOCKED (connection failed or timed out)

### Test 3: other host (blocked)
- Run from: bertil-HP-EliteBook-840-G8-Notebook-PC, Mon Sep 28 01:57:01 PM CAT 2026
- Command: nc -zv -w 3 10.0.30.1 8080
- Expected: Connection times out
- Actual output: nc: connect to 10.0.30.1 port 8080 (tcp) timed out: Operation now in progress
- Actual result: BLOCKED (connection failed or timed out)

## Firewall rule counters after the tests

Command: sudo ip netns exec server iptables -L INPUT -n -v

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
 0 0 ACCEPT 0 -- lo * 0.0.0.0/0 0.0.0.0/0
 3 156 ACCEPT 0 -- * * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
 3 180 DROP 0 -- * * 10.0.20.0/24 0.0.0.0/0
 1 60 ACCEPT 6 -- * * 10.0.10.0/24 0.0.0.0/0 tcp dpt:8080
 3 180 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8080

```

5 Conclusion

The project assessed three main risks, protected files with authenticated encryption and SHA-256 integrity checking, and restricted access to the records server with firewall rules that block guests and unauthorised hosts while allowing staff. Limitations: the firewall protects only one service, and password and update weaknesses need policy changes and patching that were not implemented here. Further work: multi-factor authentication, automatic patching, encrypted transfer protocols such as SFTP or TLS, and central logging.

GitHub repository: <https://github.com/Bertil-githu/cryptography-network-security-exam>